USER GUIDE

Delivery Insights

App walkthrough: what it is for, how it works, and how to use it page by page

PREPARED BY

Subash S

DATE

22 October 2026

AUDIENCE

Team members, team leads, managers and admins

Contents

Part 1 Getting started

1 What is Delivery Insights for? 3

2 How it works 5

3 Before you start: sign-in and roles 6

4 How to approach the app 7

Part 2 Walkthrough, page by page

5 Step 1 Overview: how is work flowing? 8

6 Step 2 Board: where is each item? 10

7 Step 3 Work item detail: why did this item take so long? 11

8 Step 4 Record a delay reason 13

9 Step 5 Sprints: did we deliver what we committed? 14

10 Step 6 Delays: what is behind, and why? 15

11 Step 7 People: is anyone overloaded or stuck? 16

12 Step 8 Admin: keep the data healthy and accountable 17

Part 3 Reference

13 Metric cheat sheet and FAQ 18

How to read this guide

- **New to the app?** Read Part 1, then follow the steps in Part 2 in order.
- **Looking for one page?** Jump straight to its step. Each chapter starts on a new page.
- **Not sure what a number means?** Part 3 explains every metric in plain words.

Select any entry above, or use the bookmarks panel in your PDF viewer, to jump to it.

CHAPTER 1 · GETTING STARTED

What is Delivery Insights for?

Your team already plans and tracks its work in Azure DevOps: the stories, bugs and sprints. Azure DevOps is good at showing what is happening **right now**. It is not good at answering the questions people ask afterwards: *How long did that really take? What changed along the way? Why was it late?*

Delivery Insights answers those questions. Think of it as a **fitness tracker for how your team delivers work**. It doesn't do the work for you, and it doesn't judge anyone. It quietly watches what already happens in Azure DevOps and shows you the patterns, so conversations about delivery start from facts instead of memory.

Who it helps

- **Team members** see their own work and can explain, in a sentence, why something got delayed.
- **Team leads** spot items that are stuck before the sprint ends, and run retrospectives with real numbers.
- **Managers** see trends across teams without chasing people for status updates.
- **Admins** keep the data flowing and can prove nobody has tampered with the records.

What it can do

The question you have	What the app shows you	Where
How long does our work usually take?	Average time from starting an item to finishing it, and from asking for it to getting it	Overview
Where is work getting stuck right now?	Every item on a board, with how long it has sat in each stage	Board
Did we finish what we promised this sprint?	Items planned vs items done, sprint by sprint	Sprints
What changed after we started?	Every change to an item: new scope, re-estimates, moved sprints, new owners	Work item detail
Which items are late, and why?	Items the app flags as delayed, plus the reason your team gave	Delays
Are our estimates realistic?	The effort we guessed compared with what it actually took	Overview, Work item detail
Is the workload balanced?	How much each person has in progress and has finished	People
Can we trust these records?	A tamper-proof log of who viewed or changed what, with a one-click check	Admin

What it doesn't do

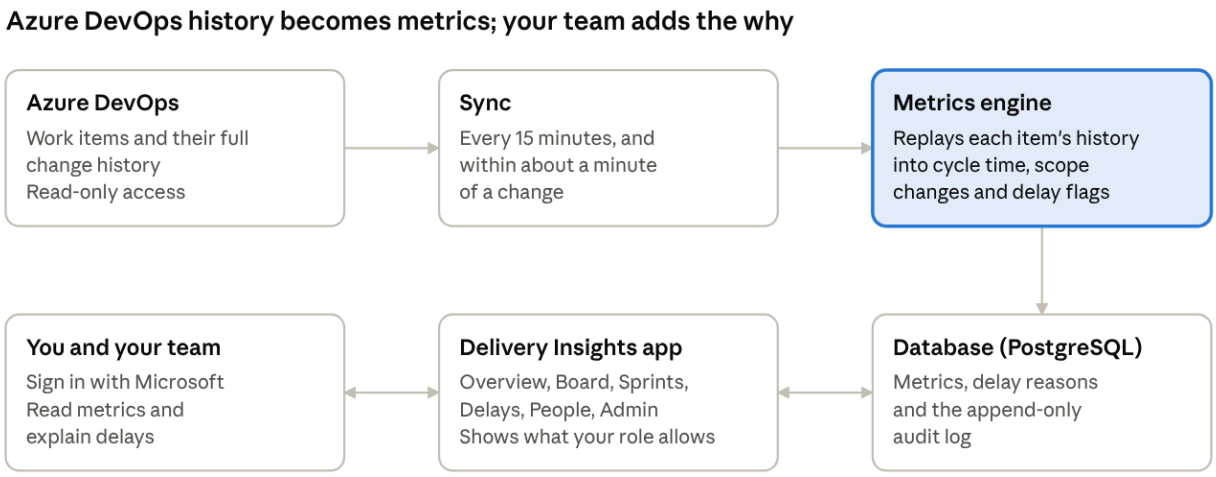
- **It never changes anything in Azure DevOps.** It only reads. You keep working in Azure DevOps exactly as you do today.
- **It isn't a tool for rating people.** The numbers describe how work flows, not how good someone is.
- **It adds almost no extra work.** The only thing anyone types in is a short delay reason, when an item runs late.
- **It's only as accurate as your board.** If a card isn't moved in Azure DevOps, the app can't know it moved. Keeping the board up to date keeps the numbers honest.

Terms such as *cycle time* and *lead time* are explained in plain words in the **Metric cheat sheet** at the end.

CHAPTER 2 · GETTING STARTED

How it works

Delivery Insights reads every work item's full change history from Azure DevOps, turns it into flow metrics, and lets your team record why items were delayed. Azure DevOps stays the source of truth: the app never writes anything back to it.



How Delivery Insights works · data flow

Because the metrics come from history rather than from the current state, every number can be traced to the exact change behind it. Delay reasons are the one thing people add, and they live only in Delivery Insights.

CHAPTER 3 · GETTING STARTED

Before you start: sign-in and roles

You sign in with your Microsoft work account (Entra ID, with your organisation's MFA). There is no separate password. You see only the Azure DevOps projects you can already see in Azure DevOps.

What you can do depends on the role your administrator assigned to you in Entra ID:

Role	Overview, Board, Sprints, Delays	Add a delay reason	People (per-person metrics)	Admin
Viewer	Yes	Only on items assigned to you	No	No
Team Lead	Yes	Yes	Yourself and your team	No
Manager	Yes	Yes	Everyone in the project	No
Admin	Yes	Yes	Everyone	Yes

- Sessions end after 30 minutes without activity, and after 8 hours at most. Sign in again to continue.
- Opening someone's individual metrics is recorded in the audit log.
- On a local test copy, the sign-in page instead offers four test personas: Ada Admin, Meera Manager, Raj Singh (Team Lead) and Alex Brown (Viewer).

CHAPTER 4 · GETTING STARTED

How to approach the app

Start wide, then drill into the items that explain the numbers. The first time, go through the pages in this order:

1. **Overview**: how fast work flows and how many items are delayed.
2. **Delays**: which items are behind, and which still have no reason recorded.
3. **Work item detail**: open one delayed item and read its story (states, scope changes, blocked time).
4. **Record a delay reason** while the cause is still fresh.
5. **Sprints**: commitment versus delivery, sprint by sprint.
6. **People** (Team Leads and Managers): load and flow per person, used for support, not ranking.
7. **Admin** (Admins only): sync health, delay categories and the audit log.

Once it is part of the routine, use it at four moments:

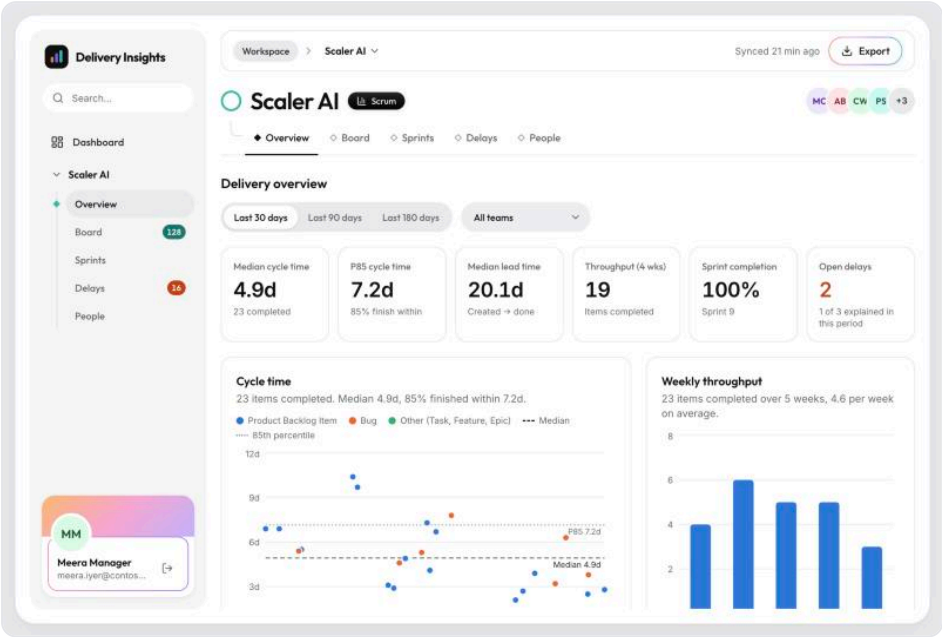
When	Who	Page	What to do
Daily stand-up	Team	Board, Delays	Spot newly flagged items and add a reason the same day
Sprint review	Team Lead, Product Owner	Sprints	Compare committed with completed; discuss items added mid-sprint
Retrospective	Team	Overview (delay reasons), Delays	Pick the top delay category and agree one action
Monthly	Manager	Overview, People	Track cycle time and throughput trends; rebalance load

The numbers describe the system of work, not individual performance. Use them to ask better questions, not to rank people.

CHAPTER 5 · STEP 1 OF 8 · WALKTHROUGH, PAGE BY PAGE

Overview: how is work flowing?

The Overview answers one question: how fast does work move, and what is slowing it down? It opens on the last 90 days for all teams.



Overview page with six KPI tiles, cycle time chart and weekly throughput

Controls along the top

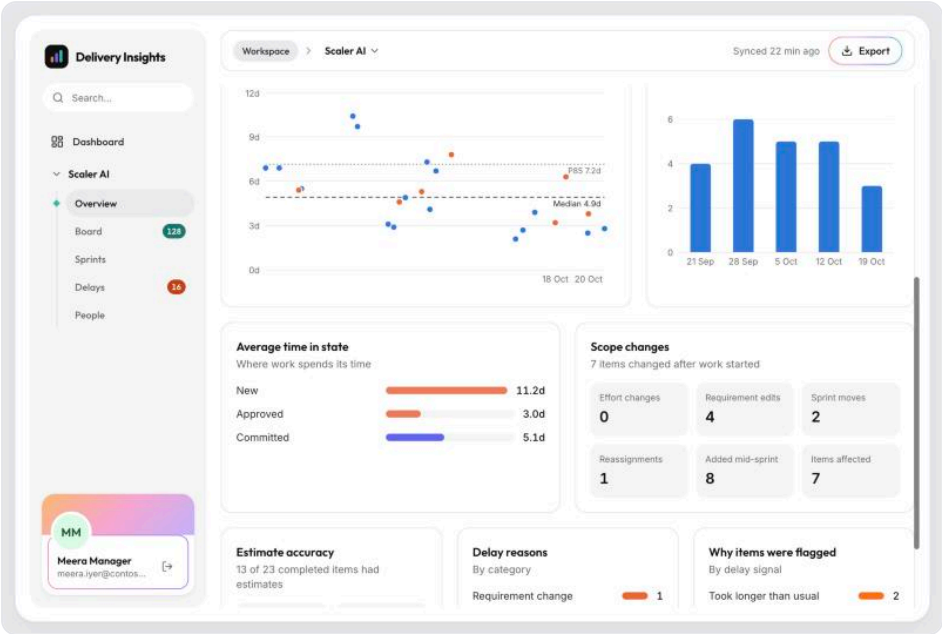
- **Period**: Last 30, 90 or 180 days. Every tile and chart on the page follows it.
- **Team**: All teams, or one team (for example Web Squad).
- **Export** (top right): downloads the table behind the current view as CSV.
- **Synced … ago**: when the data was last refreshed from Azure DevOps.

The six tiles

Tile	Read it as
Median cycle time	Half of the items took less than this from start to done.
P85 cycle time	85% finished within this. Use it when someone asks "when will it be done?"
Median lead time	From creation to done: includes waiting in the backlog.
Throughput (4 wks)	Items completed in the last four weeks.
Sprint completion	Share of committed items finished in the last finished sprint.
Open delays	Delayed items with no reason yet, and how many are explained in this period.

The charts

- **Cycle time**: one dot per completed item, coloured by type. The dashed line is the median, the dotted line the P85. Dots high above the dotted line are worth opening.
- **Weekly throughput**: items finished per week. A steady bar height is healthier than spikes.



Lower half of the Overview: time in state, scope changes, estimate accuracy, delay reasons

Scroll down for the reasons behind the numbers:

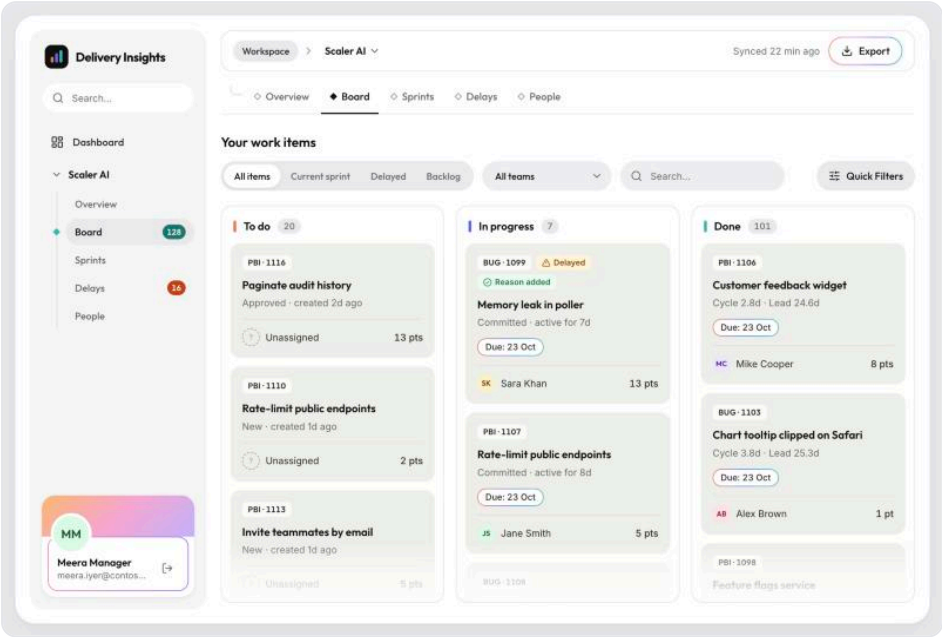
- **Average time in state**: where work waits. A long *New* bar means items wait in the backlog, not in development.
- **Scope changes**: effort changes, requirement edits, sprint moves and reassignments after work started, plus items added mid-sprint.
- **Estimate accuracy**: actual hours ÷ estimate; 1.26× means work took 26% longer than estimated.
- **Delay reasons** and **Why items were flagged**: the recorded causes, and which delay signals fired.

Every chart also has a table version for screen readers and for copying.

CHAPTER 6 · STEP 2 OF 8 · WALKTHROUGH, PAGE BY PAGE

Board: where is each item?

The Board shows every work item in three columns, so you can see at a glance what is waiting, moving and done. It mirrors Azure DevOps but adds delay signals and timings to each card.



Board page with To do, In progress and Done columns

- **Columns**: To do (New, Approved), In progress (Committed) and Done, each with its item count.
- **Views**: All items, Current sprint, Delayed or Backlog.
- **Filters**: team, search by ID or title, and **Quick Filters** for work item type and sort order (for example Recently changed).

What a card tells you:

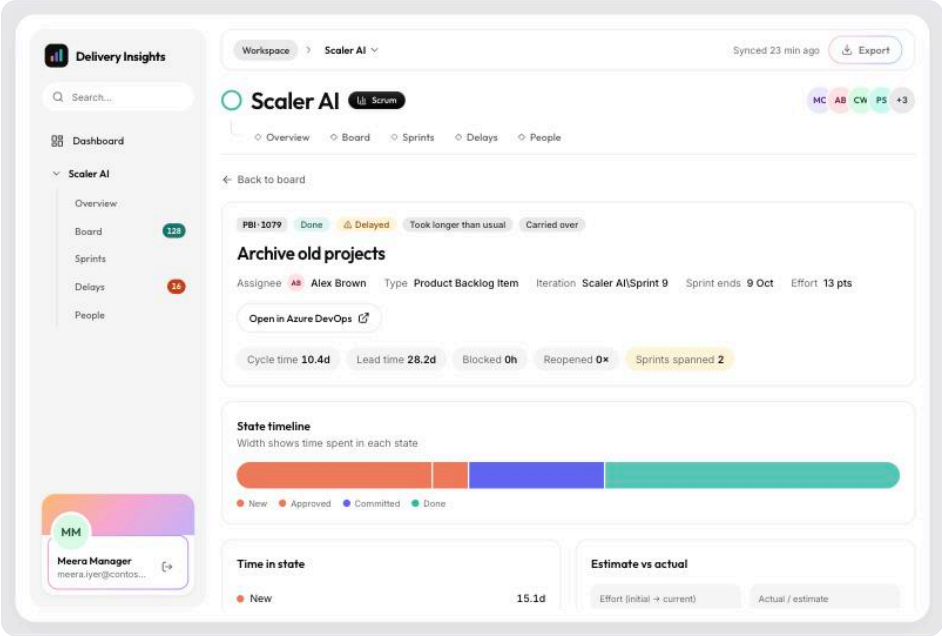
Card element	Meaning
BUG · 1099	Type and Azure DevOps ID
Delayed (amber)	At least one delay flag applies
Reason added (green)	Someone has already explained the delay
Committed · active for 7d	Current state and how long since work started
Cycle 2.8d · Lead 24.6d	On done items: how long the work took and how long the requester waited
Due: 23 Oct	End date of the item's sprint
Avatar, 13 pts	Assignee and effort

Click a card to open its full story (Step 3).

CHAPTER 7 · STEP 3 OF 8 · WALKTHROUGH, PAGE BY PAGE

Work item detail: why did this item take so long?

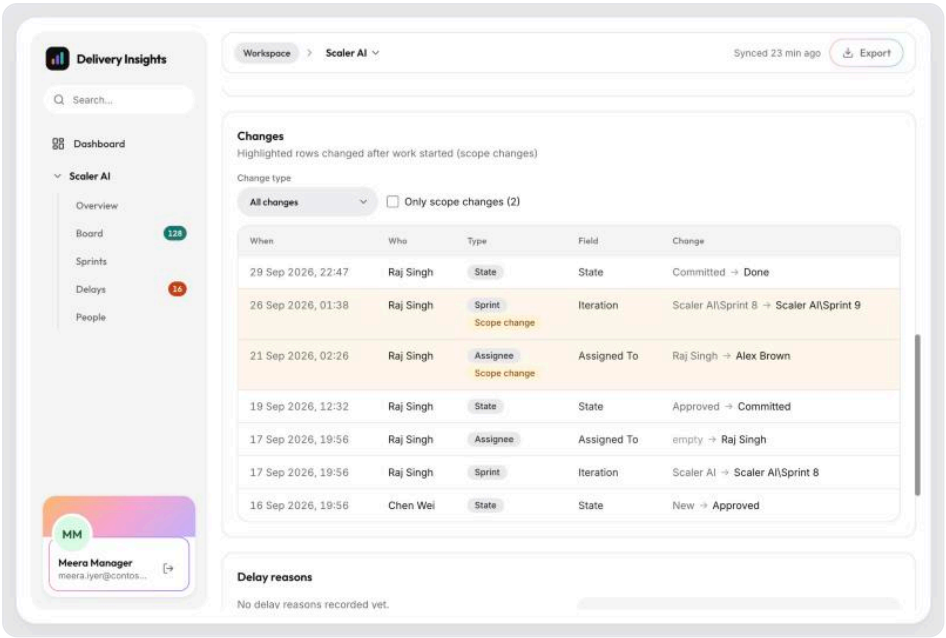
The detail page tells one item's full story from its Azure DevOps history: every state it passed through, how long it sat there and what changed along the way. In the example below, PBI 1079 took 10.4 days of work but 28.2 days end to end, and moved across two sprints.



Work item detail for PBI 1079 with flags, timings and state timeline

Read it from top to bottom:

1. **Header badges**: type and ID, current state, *Delayed*, and each delay flag that fired (here *Took longer than usual* and *Carried over*).
2. **Facts**: assignee, type, sprint, sprint end date and effort. **Open in Azure DevOps** jumps to the original item.
3. **Timing chips**: cycle time, lead time, blocked time, times reopened, and sprints spanned (highlighted when it is more than one).
4. **State timeline**: one bar per state, its width proportional to the time spent there. Hover a segment for its exact dates.
5. **Time in state** and **Estimate vs actual**: the same time as numbers, and effort initial → current, actual ÷ estimate, original estimate and completed work.
6. **Description** and **acceptance criteria**, shown as plain text.



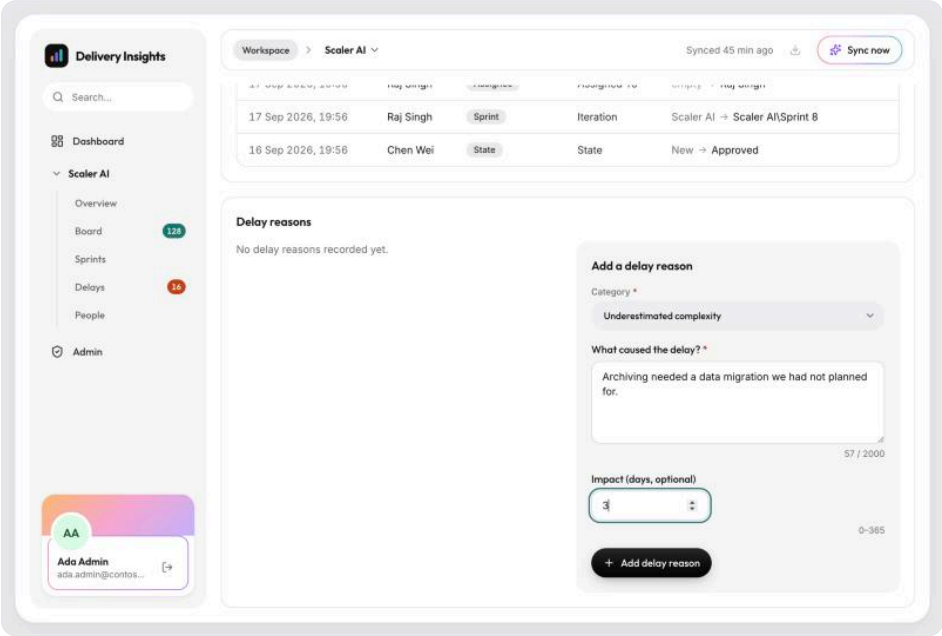
Changes table with two scope changes highlighted

The **Changes** table lists every field change, newest first, with who made it and when. Rows highlighted in amber and marked *Scope change* happened after work started: a sprint move, a reassignment, an effort change or a requirement edit. Use **Change type** or **Only scope changes** to filter. Here the item was reassigned on 21 Sep and moved from Sprint 8 to Sprint 9 on 26 Sep, which explains the carry-over.

CHAPTER 8 · STEP 4 OF 8 · WALKTHROUGH, PAGE BY PAGE

Record a delay reason

The app detects *that* an item is delayed; only the team knows *why*. Recording the reason turns a red flag into something the team can learn from in the retrospective. Do it the same day, while the cause is fresh.



Add a delay reason form filled in with a category, explanation and 3 days impact

1. Open the item, from the Board or with **Add reason** on the Delays page (it jumps straight to this form).
2. **Category**: pick the closest cause, such as *Dependency on another team*, *Requirement change* or *Underestimated complexity*. Admins maintain this list.
3. **What caused the delay?**: at least 10 characters, up to 2,000. Be specific: *Waiting on the payments API contract from another team* helps; *blocked* does not.
4. **Impact (days, optional)**: your best estimate of the time lost, from 0 to 365.
5. Select **Add delay reason**. The reason appears above the form with your name, the time and a *Current* badge.

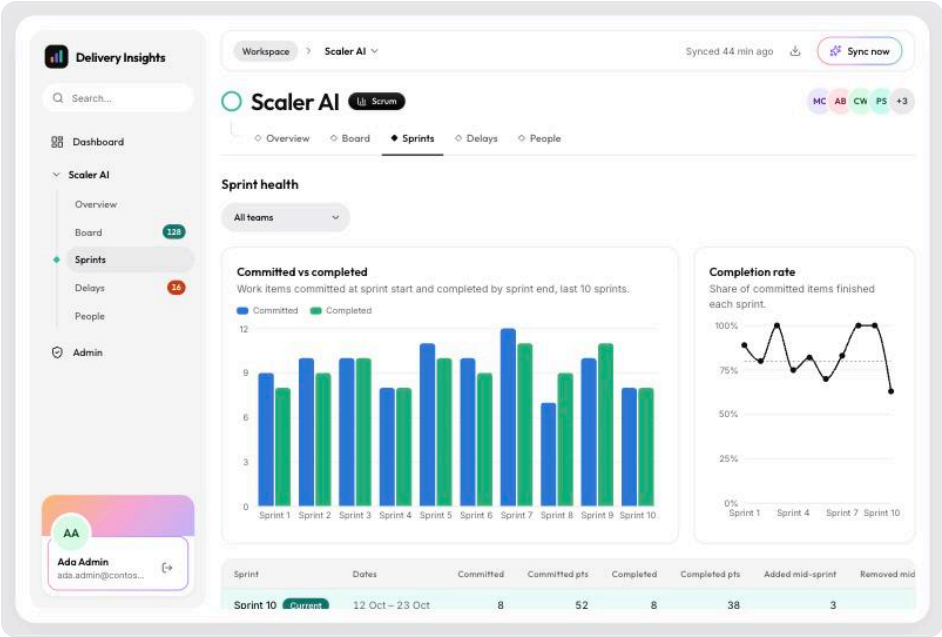
Who can add one: the item's assignee, Team Leads, Managers and Admins.

Reasons can't be edited or deleted. If the cause changes, add a new reason: it becomes *Current*, and the earlier one stays in the history. Every reason is also written to the audit log.

CHAPTER 9 · STEP 5 OF 8 · WALKTHROUGH, PAGE BY PAGE

Sprints: did we deliver what we committed?

The Sprints page compares what each sprint committed to with what it finished, for the last 10 sprints. Use it in sprint review and planning.



Sprints page with committed vs completed bars and the completion rate line

- **Committed vs completed**: blue bars are items in the sprint at its start (with one day's grace for planning), green bars are items finished during it.
- **Completion rate**: committed items finished ÷ committed items, per sprint.
- **Sprint health table** (below the charts), newest first: dates, committed and completed counts and points, added mid-sprint, removed mid-sprint, carried over, and completion. The running sprint is marked *Current*.
- **Team** filter: one team, or all.

Two things that surprise people at first:

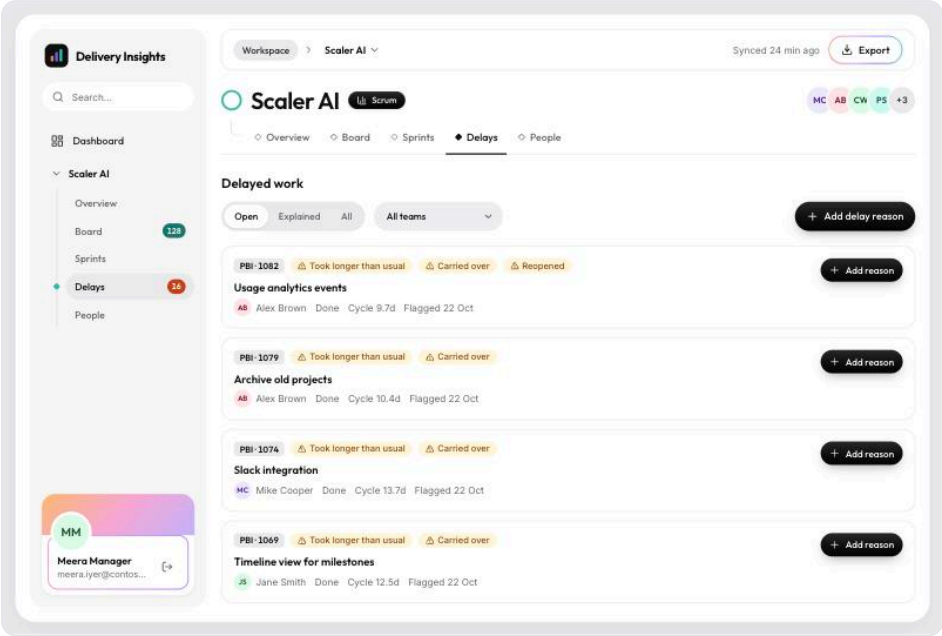
- **Completed can be higher than committed.** In Sprint 9 the team committed 10 items and finished 11, because one was added mid-sprint. Completion stays at 100%: added work never inflates the rate.
- **The current sprint looks low** (Sprint 10: 63%) because it isn't over yet. The Overview tile therefore uses the last finished sprint.

What to look for: a falling completion rate, many items added mid-sprint, or the same items carried over sprint after sprint. Each points to a planning conversation.

CHAPTER 10 · STEP 6 OF 8 · WALKTHROUGH, PAGE BY PAGE

Delays: what is behind, and why?

The Delays page is the team's to-do list for explanations: every item the app flagged, with the signals that fired. The red number next to **Delays** in the sidebar is the count still waiting for a reason.



Delays page listing flagged items with their delay signals and Add reason buttons

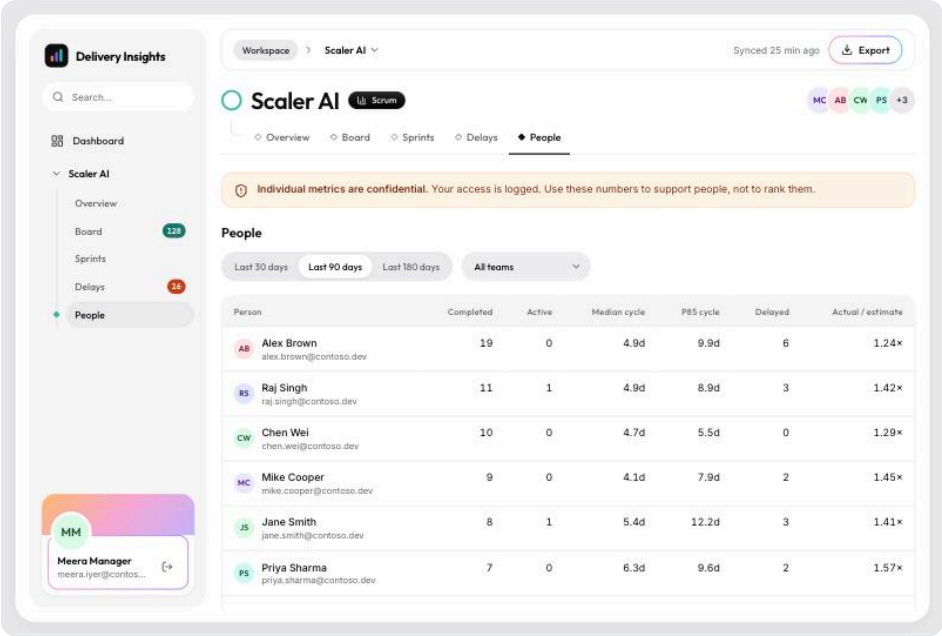
- **Open / Explained / All**: open items have no reason yet; explained items show the latest reason's category, author, date and comment.
- **Team** filter: one team, or all.
- **Each row**: type and ID, the delay signals (for example *Took longer than usual*, *Carried over*, *Reopened*), title, assignee, state, cycle time and the date it was flagged.
- **Add reason** (or **Update reason** once explained) opens the item at its delay-reason form.
- **Add delay reason** (top right) lets you pick any delayed item by ID, with unexplained items suggested.

Work the list until **Open** shows *No unexplained delays*. The retrospective then has a complete picture of why work slipped.

CHAPTER 11 · STEP 7 OF 8 · WALKTHROUGH, PAGE BY PAGE

People: is anyone overloaded or stuck?

The People page shows delivery metrics per person, so Team Leads and Managers can spot someone who is overloaded, blocked or repeatedly given work that slips. It is restricted and every visit is logged.



People page with the confidentiality banner and per-person metrics table

Column	Meaning
Completed	Items this person finished in the period
Active	Items currently in progress
Median cycle, P85 cycle	Their typical and 85th-percentile time from start to done
Delayed	Their items that were flagged as delayed
Actual / estimate	Average actual hours ÷ estimate; above 1× means work took longer than estimated

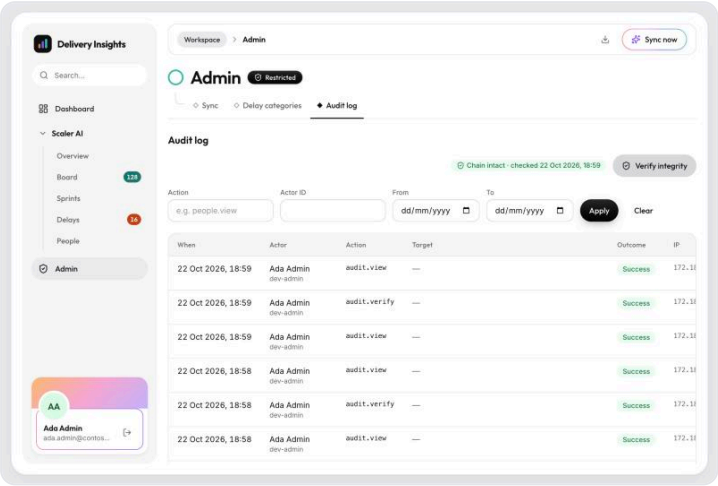
- **Who can open it**: Team Leads (themselves and their team), Managers and Admins. Viewers don't see this page.
- **Every visit is recorded** in the audit log as *people.view*, with who looked and which project.
- Use the same **period** and **team** filters as on the Overview.

How to use it well: open it in one-to-ones, ask what got in the way and look at the delay reasons together. A delay often has a cause outside the person, such as a dependency or an unclear requirement. Don't compare people's numbers against each other.

CHAPTER 12 · STEP 8 OF 8 · WALKTHROUGH, PAGE BY PAGE

Admin: keep the data healthy and accountable

The Admin area (Admins only, marked *Restricted*) has three tabs: Sync, Delay categories and Audit log.



Audit log tab with filters and the Verify integrity check

Sync

- One row per sync run: status, start time, duration, trigger (manual, scheduled, change notification, startup), project, items and revisions read, and the error if it failed.
- **Run sync** fetches the latest changes now; **Full resync** re-reads the whole history window. Pick one project or all. **Sync now** in the header starts a sync of the project you are viewing.
- A failed run says why, for example *Azure DevOps GET …/_apis/projects failed: 404* when the organisation URL is wrong.

Delay categories

- Add a category, or edit one's name (up to 80 characters), description (up to 400) and whether it is **Active**.
- An inactive category disappears from the delay-reason form, but reasons that already use it keep it.

Audit log

- Records sign-in and sign-out, denied access, People views, delay reasons, category changes, sync triggers, audit-log views and retention runs, with who, when, the target, the outcome and the IP address.
- Filter by action (for example *people.view*), actor ID and date range.
- The log is append-only, and each entry is chained to the previous one with a SHA-256 hash, so any edit or deletion is detectable. Select **Verify integrity** (top right) to check the whole chain: a green *Chain intact* badge means no entry was altered or removed.

CHAPTER 13 · REFERENCE

Metric cheat sheet and FAQ

Every number comes from the item's change history in Azure DevOps, so any figure can be traced back to the exact change behind it. For Scrum, New and Approved count as not started, Committed as in progress, and Done as completed.

Metric	What it means
Cycle time	From the first time work started (Committed) to Done. How long the work took.
Lead time	From creation to Done. How long the requester waited.
P85	85% of items finish within this many days. A realistic commitment figure.
Throughput	Items completed per week (weeks start Monday).
Time in state	Time spent in each state, such as New, Approved or Committed.
Blocked time	Time with Blocked = Yes or a tag starting with "Blocked".
Scope change	A change to effort, requirement text, sprint or assignee after work started.
Added mid-sprint	Entered a sprint more than one day after it started.
Completion rate	Committed items completed ÷ committed items. Added work does not inflate it.
Estimate accuracy	Completed Work ÷ Original Estimate; between 0.8 and 1.2 counts as on target.

An item is **delayed** when at least one of these flags applies. Flags are recalculated after every sync:

Flag (as shown in the app)	Rule
Took longer than usual	Cycle time, or age if still open, is above the P85 of the last 180 days for that type (needs 5 completed items).
Carried over	Still unfinished in a sprint when that sprint ended.
Past sprint end	Not done, and its current sprint ended more than a day ago.
Blocked a long time	Blocked for more than 48 hours.
Reopened	Moved back from Done to an earlier state at least once.

FAQ

- **Why don't my numbers match an Azure DevOps query?** Queries show the current state; this app replays history. A reopened item, for example, keeps its first start date.
- **Why is estimate accuracy empty?** The Scrum process has no Original Estimate or Completed Work fields on backlog items by default. Add them, or log hours on Tasks.
- **How fresh is the data?** In Azure, changes arrive within about a minute through change notifications, with a full catch-up every 15 minutes. The header shows the last sync time.
- **Can a delay reason be edited?** No. Add a new reason; it replaces the current one and the history is kept.